CHAPTER 7
ARITHMETIC AND COMPARATOR CIRCUITS

7.1 Introduction

Digital components perform a variety of information-processing tasks. Among the basic functions encountered are the various arithmetic operations and comparison. Digital circuits performing these operations are combinational circuits. These circuits may be designed using the design procedure for combinational logic presented in Chapter 6. This classical procedure assumes that, given two circuits that perform the same function, the one that requires fewer gates is preferable because it will cost less. This is not necessarily true when integrated circuits are used. Since several logic gates are included in a single IC package, it becomes economical to use as many of the gates from an already used package even if, by doing so, we increase the total number of gates. Moreover, some of the interconnections among gates in many ICs are internal to the chip and it is more economical to use as many internal interconnections as possible in order to minimize the number of wires between external pins. With integrated circuits, it is not the count of gates that determines the cost but the number and type of ICs employed and the number of external interconnections needed to implement the given function.

There are numerous occasions where the classical method of Chapter 6 will not produce the best combinational circuit for implementing a given function. Moreover, the truth table and the simplification procedure in this method become too cumbersome if the number of input variables is excessively large. The final circuit obtained dictates that it be implemented with a random connection of SSI gates, which may require a relatively large number of ICs and interconnecting wires. In many cases the application of an alternate design procedure can produce a combinational circuit for a given function which is far better than the one obtained by following the classical design method. The possibility of an alternate design procedure depends on the particular problem and the ingenuity of the designer. The classical method constitutes a general procedure that, if followed, guarantees to produce a result. However, before applying the classical method, it is always wise to investigate the possibility of an alternate method that may be more efficient for the particular problem at hand.

The first question that must be answered before going through a detailed design of a combinational circuit is whether the function is already available in an IC package. Numerous MSI devices are available commercially. These devices perform specific digital functions commonly employed in the design of digital systems. If an MSI device cannot be found to produce exactly the function needed, a resourceful designer may be able to formulate a method so as to incorporate an MSI device in this circuit. The selection of MSI components in preference to SSI gates is extremely important, since it would invariably result in a considerable reduction of IC packages and interconnecting wires.

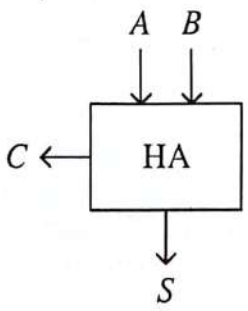
In this chapter, the basic functions are designed using classical design method and some are designed using nonclassical design method. The more complicated functions are designed by incorporating MSI devices. After going through the design examples, a reader will be familiar with when to use classical method and when to uses nonclassical method.

7.2 Adders

The most basic arithmetic operation is the addition of two binary digits. This simple addition consists of four possible elementary operations, namely, 0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1, and 1 + 1 = 10. The first three operations produce a sum whose length is one bit, but when both augend and addend bits are equal to 1, the binary sum consists of two bits. The higher significant bit of this result is called a *carry*. When the augend and addend numbers contain more significant bits, the carry obtained from the addition of two is added to the next higher-order pair of significant bits. A combinational circuit that performs the addition of two bits is called a *half-adder*. One that performs the addition of three bits (two significant bits and a previous carry) is a *full-adder*. The name of the former stems from the fact that two half-adders can be employed to implement a full-adder. The design of two adder circuits using classical method is presented next.

7.2.1 Half-Adder

From the verbal explanation of a half-adder, we find that this circuit needs two binary inputs and two binary outputs. The input variables designate the augend and addend bits; the output variables produce the sum and carry. We arbitrarily assign A and B to the two inputs and S (for sum) and C (for carry) to the outputs. The block diagram and the truth table of the half-adder are shown in Figure 7.1.



(a) Block diagram

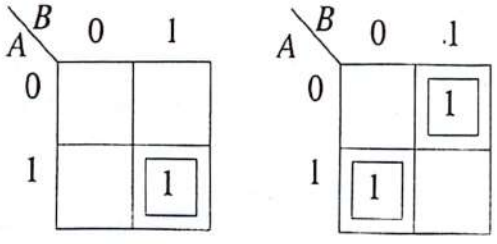
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

(b) Truth table

Figure 7.1 Block diagram and truth table of the half-adder.

The K-map simplification of the output functions of the half-adder is shown in Figure 7.2, which gives

$$S = A'B + AB' = A \oplus B$$
$$C = AB$$



(a) $C = AB$ (b) $S = A'B + AB'$

Figure 7.2 K-map simplification of output functions of the half-adder.

The implementation of the half-adder using AND-OR gates is shown in Figure 7.3(a) and using EXOR gate is shown in Figure 7.3(b).

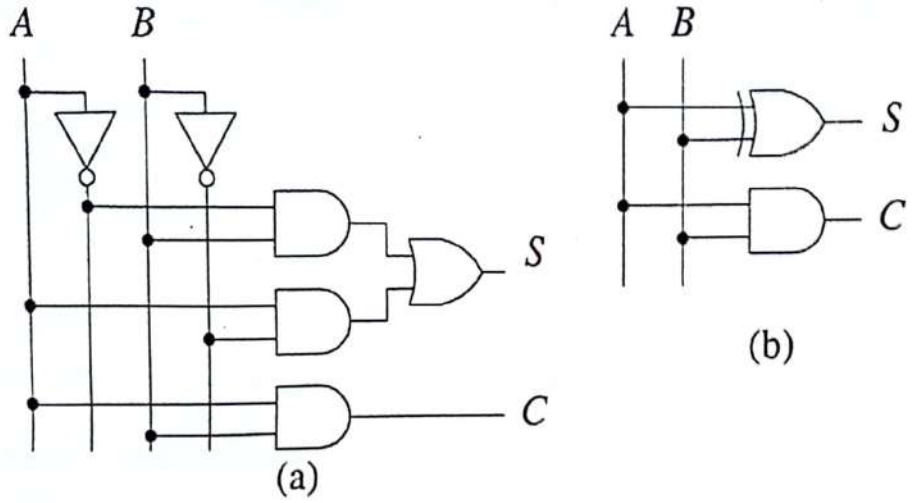
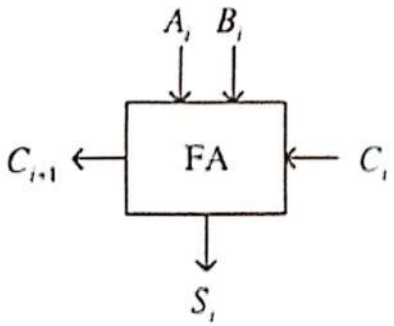


Figure 7.3 Implementation of half-adder.

7.2.2 Full-Adder

A full-adder is a combinational circuit that forms the arithmetic sum of three input bits. It consists of three inputs and two outputs. The block diagram of the full-adder is shown in Figure 7.4(a). Two of the input variables, denoted by A_i and B_i, represent the two significant bits to be added. The third input, C_i, represents the carry input from the previous lower significant position. The two outputs are designated by the symbols S_i for sum and C_{i+1} for carry output to the next higher significant position. The truth table of the full-adder is shown in Figure 7.4(b).



(a) Block diagram

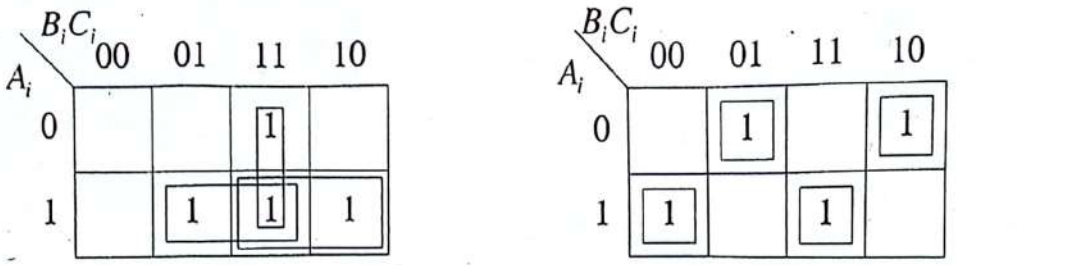
A_i	B_i	C_i	C_{i+1}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(b) Truth table

Figure 7.4 Block diagram and truth table for the full-adder.

The K-map simplification of the output functions of the full-adder is shown in Figure 7.5, which gives

$$S_i = A_i'B_i'C_i + A_i'B_iC_i' + A_iB_i'C_i' + A_iB_iC_i$$
$$C_{i+1} = A_iB_i + A_iC_i + B_iC_i$$



(a) $C_{i+1} = A_iB_i + A_iC_i + B_iC_i$ (b) $S_i = A_i'B_i'C_i + A_i'B_iC_i' + A_iB_i'C_i' + A_iB_iC_i$

Figure 7.5 K-map simplification of output functions of the full-adder.

The logic diagram for the full-adder implementation is shown in Figure 7.6.

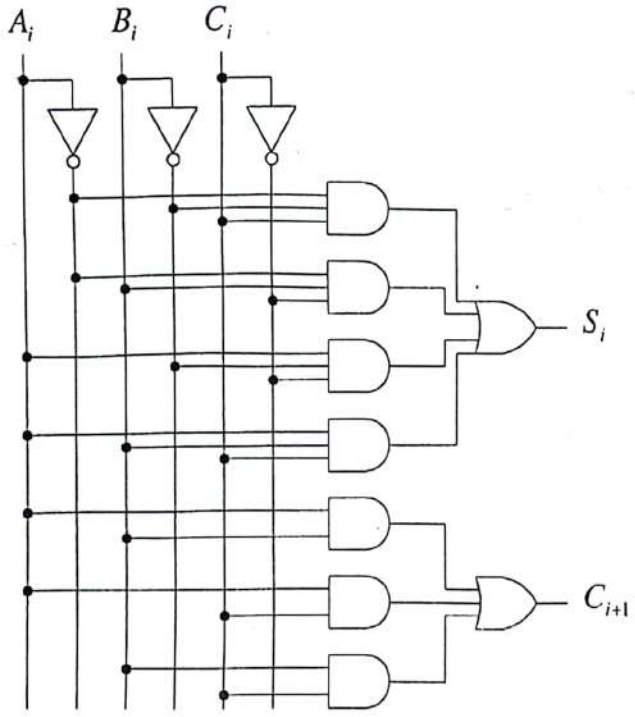
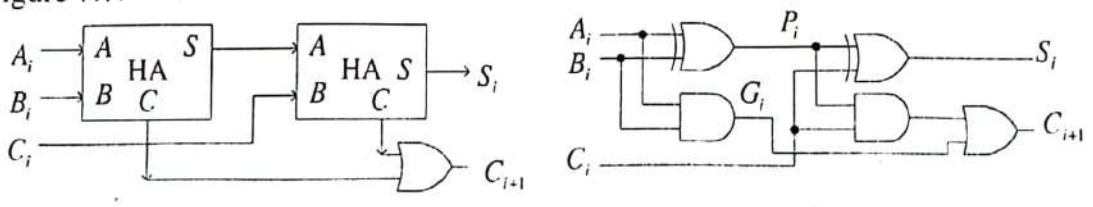


Figure 7.6 Implementation of the full-adder.

7.2.3 Full-Adder using Half-Adder

A full-adder can be implemented with two half-adders and one OR gate, as shown in Figure 7.7.



(a) Connection using block diagram (b) Logic diagram

Figure 7.7 Implementation of a full-adder with two half-adders and an OR gate.

The output functions can be verified as follows:

$$P_i = A_i \oplus B_i$$
$$G_i = A_i B_i$$

The expression for S_i is

$$\begin{aligned} S_i &= P_i \oplus C_i \\ &= P_i C_i' + P_i' C_i \\ &= (A_i \oplus B_i) C_i' + (A_i \oplus B_i)' C_i \\ &= (A_i B_i' + A_i' B_i) C_i' + (A_i B_i + A_i' B_i') C_i \\ &= A_i B_i' C_i' + A_i' B_i C_i' + A_i B_i C_i + A_i' B_i' C_i \end{aligned}$$

The expression for C_{i+1} is

$$\begin{aligned} C_{i+1} &= G_i + P_i C_i \\ &= A_i B_i + (A_i \oplus B_i) C_i \\ &= A_i B_i + (A_i B_i' + A_i' B_i) C_i \\ &= A_i B_i + A_i B_i' C_i + A_i' B_i C_i \\ &= A_i B_i (C_i + C_i') + A_i B_i' C_i + A_i' B_i C_i \\ &= (A_i B_i C_i + A_i B_i C_i') + (A_i B_i C_i + A_i B_i' C_i) + (A_i B_i C_i + A_i' B_i C_i) \\ &= A_i B_i + A_i C_i + B_i C_i \end{aligned}$$

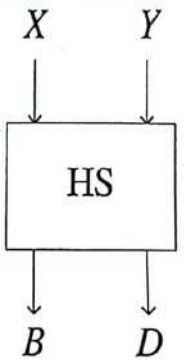
These expressions verify the correctness of the full-adder implementation of Figure 7.7.

7.3 Subtractor

The subtraction of two binary numbers may be accomplished by taking the complement of the subtrahend and adding it to the minuend (Section 2.5). By this method, the subtraction operation becomes an addition operation requiring full-adder for its machine implementation. It is, however, possible to implement subtraction with logic circuits in a direct manner, as done with paper and pencil. By this method, each subtrahend bit of the number is subtracted from its corresponding significant minuend bit to form a difference bit. If the minuend bit is smaller than the subtrahend bit, a 1 is borrowed from the next significant position. The fact that a 1 has been borrowed must be conveyed to the next higher pair of bits by means of a binary signal coming out (output) of a given stage and going into (input) the next higher stage. Just as there are half- and full-adders, there are half- and full-subtractors.

7.3.1 Half-Subtractor

A half-subtractor is a combinational circuit that subtracts two bits and produces their difference. It also has an output to specify if a 1 has been borrowed. The block diagram of the half-subtractor is shown in Figure 7.8(a). We arbitrarily designate the minuend bit by X and the subtrahend bit by Y. To perform $X - Y$, we have to check the relative magnitudes of X and Y. If $X \geq Y$, we have three possibilities: $0 - 0 = 0$, $1 - 0 = 1$, and $1 - 1 = 0$. The results is called the *difference bit*. If $X < Y$, we have $0 - 1$, and it is necessary to borrow a 1 from the next higher stage. The 1 borrowed from the next higher stage adds 2 to the minuend bit. With the minuend equal to 2, the difference becomes $2 - 1 = 1$. The half-subtractor needs two outputs. One output generates the difference and will be designated by the symbol D. The second output, designated B for borrow, generates the binary signal that informs the next stage that a 1 has been borrowed. The truth table of the half-subtractor is shown in Figure 7.8(b).



(a) Block diagram

X	Y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

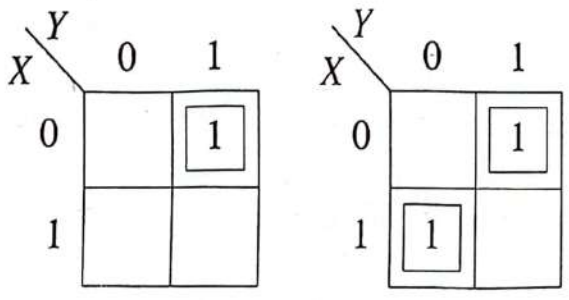
(b) Truth table

Figure 7.8 Block diagram and truth table of the half-subtractor.

The K-map simplification of the output functions of the half-subtractor is shown in Figure 7.9, which gives

$$D = X'Y + XY' = X \oplus Y$$

$$B = X'Y$$



(a) $B = X'Y$ (b) $D = X'Y + XY'$

Figure 7.9 K-map simplification of the output functions of the half-subtractor.

The implementation of the half-subtractor is shown in Figure 7.10.

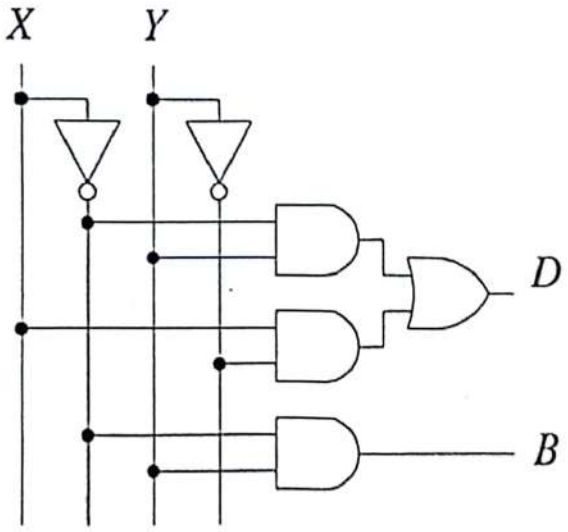
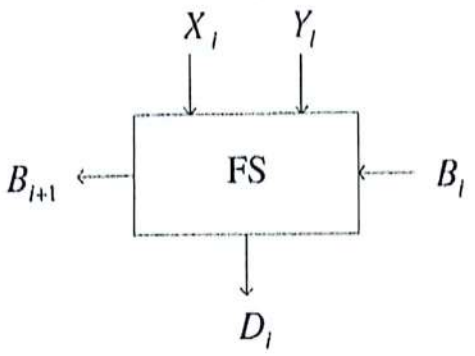


Figure 7.10 Implementation of the half-subtractor.

It is interesting to note that the logic for D is exactly the same as the logic for S output in the half-adder.

7.3.2 Full-Subtractor

A full-subtractor is a combinational circuit that performs a subtraction between two bits, taking into account that a 1 may have been borrowed by a lower significant stage. This circuit has three inputs and two outputs. The three inputs, X_i, Y_i, and B_i denote the minuend, subtrahend, and previous borrow, respectively. The two outputs, D_i and B_{i+1}, represent the difference and output borrow, respectively. The block diagram of the full-subtractor is shown in Figure 7.11(a) and its truth table is shown in Figure 7.11(b). The values of the output variables are determined from the subtraction of $X_i - Y_i - B_i$. The input combinations having input borrow $B_i = 0$ reduce to the same four conditions of the half-subtractor. For $X_i = 0$, $Y_i = 0$, and $B_i = 1$, we have to borrow a 1 from the next stage, which makes $B_{i+1} = 1$ and adds 2 to X_i. Since $2 - 0 - 1 = 1$, $D_i = 1$. For $X_i = 0$, $Y_i = 1$, and $B_i = 1$, we need to borrow again, making $B_{i+1} = 1$ and $X_i = 2$. Since $2 - 1 - 1 = 0$, $D_i = 0$. For $X_i = 1$, $Y_i = 0$, and $B_i = 1$, we have $X_i - Y_i - B_i = 0$, which makes $B_{i+1} = 0$ and $D_i = 0$. Finally, for $X_i = 1$, $Y_i = 1$, and $B_i = 1$, we have to borrow 1, making $B_{i+1} = 1$ and $X_i = 3$, and $3 - 1 - 1 = 1$, making $D_i = 1$.



(a) Block diagram

X_i	Y_i	B_i	B_{i+1}	D_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

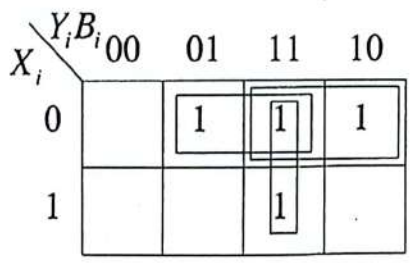
(b) Truth table

Figure 7.11 Block diagram and truth table of the full-subtractor.

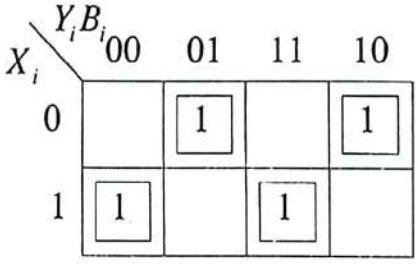
The K-map simplification of the output functions is shown in Figure 7.12, which gives

$$D_i = X_i'Y_i'B_i + X_i'Y_iB_i' + X_iY_i'B_i' + X_iY_iB_i$$

$$B_{i+1} = X_i'Y_i + X_i'B_i + Y_iB_i$$



(a) $B_{i+1} = X_i'Y_i + X_i'B_i + Y_iB_i$



(b) $D_i = X_i'Y_i'B_i + X_i'Y_iB_i' + X_iY_i'B_i' + X_iY_iB_i$

Figure 9.12 K-map simplification of the full-subtractor.

Again we note that the logic function for output D_i in the full-subtractor is exactly the same as output S_i in the full-adder. Moreover, the output B_{i+1} resembles the function for C_{i+1} in the full-adder, except that the input variable X_i is complemented. Because of these similarities, it is possible to convert a full-adder into a full-subtractor by merely complementing X_i prior to its application to the gates that form the carry output. The implementation of the full-subtractor is shown in Figure 7.13.

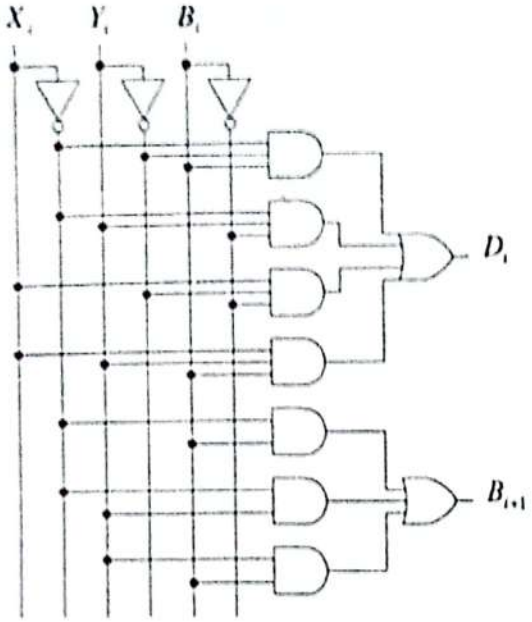


Figure 7.13 Implementation of the full-subtractor.

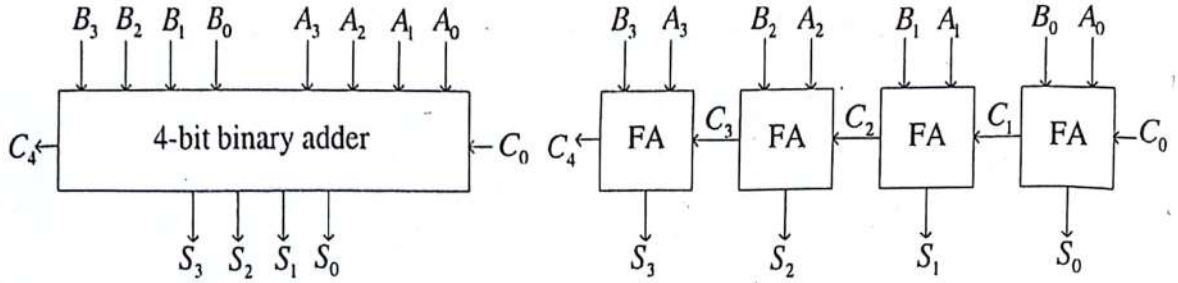
7.4 Binary Parallel Adder

The full-adder introduced in Section 7.2 forms the sum of two bits and a previous carry. Two binary numbers of n bits each can be added by means of this circuit. To demonstrate with a specific example, consider two binary numbers, $A = 1010$ and $B = 0011$, whose sum is $S = 1110$. When a pair of bits are added through a full-adder, the circuit produces a carry to be used with the pair of bits one significant position higher. This is shown below.

Subscript	3 2 1 0
Augend, A_i	1 0 1 1
Addend, B_i	0 0 1 1
Input Carry, C_i	0 1 1 0
Sum, S_i	1 1 1 0
Output Carry, C_{i+1}	0 0 1 1

The bits are added with full-adders, starting from the least significant position (subscript 0), to form the sum bit and carry bit. The input carry C_0 in the least significant position must be 0. The value of C_{i+1} is the output carry of the full-adder in the ith significant position. This value is transferred into the input carry of the full-adder that adds the bits in the $(i + 1)$st significant position, that is, one higher significant position to the left. The sum bits are thus generated starting from the rightmost position and are available as soon as the corresponding previous carry bit is generated.

A *binary parallel adder* (or *binary adder* in short) is a digital function that produces the arithmetic sum of two binary numbers in parallel. It consists of full-adders connected in cascade, with the output carry from one full-adder connected to the input carry of the next full-adder. The block diagram of a 4-bit binary adder is shown in Figure 7.14(a) and its implementation using four full-adders is shown in Figure 7.14(b).



(a) Block diagram (b) Implementation using full-adders

Figure 7.14 4-bit binary adder.

All bits of A and B are applied simultaneously. The output carry from one full-adder is connected to the input carry of the full-adder one position to its left. As soon as the carries are generated, the correct sum bits emerge from the sum outputs of all full-adders.

An n-bit binary adder requires n full-adders. It can be constructed from 4-bit, 2-bit, and 1-bit binary adder ICs by cascading several packages. The output carry from one package must be connected to the input carry of the one with the next higher-order bits. Cascading of two 4-bit binary adders to implement a 8-bit binary adder is shown in Figure 7.15. The output carry of the leftmost stage indicates the overflow, that is, a 1 in C_8 indicates that the sum exceeds 11111111.

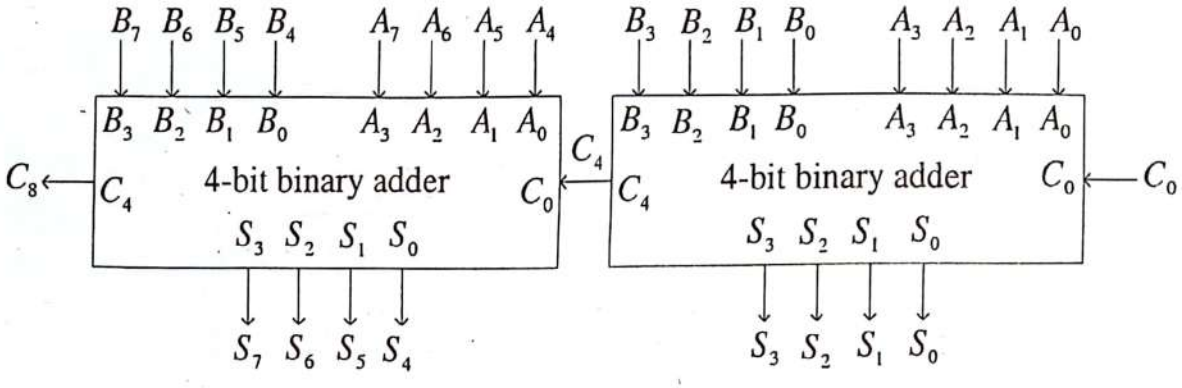


Figure 7.15 Cascading of two 4-bit binary adders to implement a 8-bit binary adder.

The 4-bit binary adder is a typical example of an MSI function. It can be used in many applications involving arithmetic operations. Observe that the design of this circuit by the classical method would require a truth table with $2^9 = 512$ entries, since there are nine inputs to the circuit. By using an iterative method of cascading an already known function, we were able to obtain a simple and well-organized implementation.

7.5 Binary Adders with Look-Ahead Carry

The addition of two binary numbers in parallel implies that all the bits of the augend and the addend are available for computation at the same time. As in any combinational circuit, the signal must propagate through the gates before the correct output sum is available in the output terminals. The total propagation time is equal to the propagation delay of a typical gate times the number of gate levels in the circuit. The longest propagation delay time in a parallel adder is the time it takes the carry to propagate through the full-adders. Since each bit of the sum output depends on the value of the input carry, the value of S_i in any given stage in the adder will be in its steady-state final value only after the input carry to that stage has been propagated. Consider output S_3 in Figure 7.14(b). Inputs A_3 and B_3 reach a steady value as soon as input signals are applied to the adder. But input carry C_3 does not settle to its final steady-state value until C_2 is available in its steady-state value. Similarly, C_2 has to wait for C_1, and so on down to C_0. Thus only after the carry propagates through all stages, the last output S_3 and carry C_4 will settle to their final steady-state value.

The number of gate levels for the carry propagation can be found from the circuit of the full-adder of Figure 7.7(b). The signals at P_i and G_i settle to their steady-state value after the propagation through their respective gates. These two signals are common to all full-adders and depend only on the input augend and addend bits. The signal from the input carry, C_i, to the output carry, C_{i+1}, propagates through an AND gate and an OR gate, which constitutes two gate levels. If there are four full-adders in the parallel adder, the output carry C_4 would have $2 \times 4 = 8$ gate levels from C_0 to C_4. The total propagation time in the adder would be the propagation time in one half-adder plus eight gate levels. For an n-bit parallel adder, there are $2n$ gate levels for the carry to propagate through.

The carry propagation time is a limiting factor on the speed with which two numbers are added in parallel. Although a parallel adder, or any combinational circuit, will always have some value at its output terminals, the outputs will not be correct unless the signals are given enough time to propagate through the gates connected from the inputs to the outputs. Since all other arithmetic operations are implemented by successive additions, the time consumed during the addition process is very critical. An obvious solution for reducing the carry propagation delay times is to employ faster gates with reduced delays. But physical circuits have a limit to their capacity. Another solution is to increase the equipment complexity in such a way that the carry delay time is reduced. There are several techniques for reducing the carry propagation time in a parallel adder. The most widely used technique employs the principle of *look-ahead* carry.

Consider the circuit of the full-adder shown in Figure 7.7(b). The output sum and carry can be expressed as

$$S_i = P_i \oplus C_i$$
$$C_{i+1} = G_i + P_i C_i$$

where

$$P_i = A_i \oplus B_i$$
$$G_i = A_i B_i$$

G_i is called a *carry generate* and it produces an output carry when both A_i and B_i are one, regardless of the input carry. P_i is called a carry propagate because it is the term associated with the propagation of the carry from C_i to C_{i+1}.

We now write the Boolean function for the carry output of each stage and substitute for each C_i its value from the previous equations.

$$C_1 = G_0 + P_0 C_0$$

$$\begin{aligned} C_2 &= G_1 + P_1 C_1 \\ &= G_1 + P_1 (G_0 + P_0 C_0) \\ &= G_1 + P_1 G_0 + P_1 P_0 C_0 \end{aligned}$$

$$\begin{aligned} C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0) \\ &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3 C_3 \\ &= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) \\ &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0. \end{aligned}$$

Since the Boolean functions for each output carry is expressed in sum of products form, each function can be implemented with one level of AND gates followed by an OR gate. The circuit implementing these functions is called *look-ahead carry generator*. The block diagram and logic implementation of 4-bit look-ahead carry generator is shown in Figure 7.16. Note that C_4 does not have to wait for C_3, C_2, and C_1 to propagate; in fact, C_4 is propagated at the same time as C_3, C_2, and C_1.

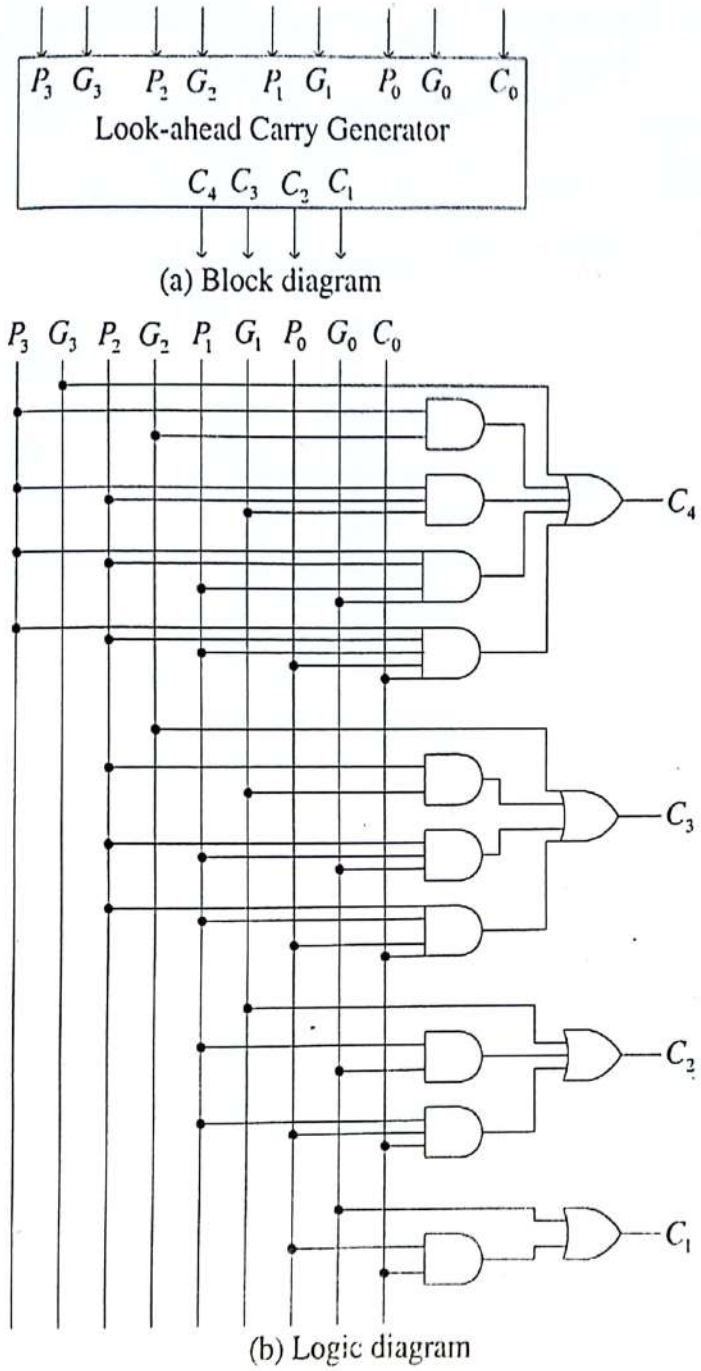


Figure 7.16 4-bit look-ahead carry generator.

The construction of a 4-bit binary adder with a look-ahead carry generator is shown in Figure 7.17. Each sum output requires two EXOR gates. The output of the first EXOR gate generates the P_i variable, and the AND gate generates the G_i variable. All the *P*s and *G*s are generated in one gate level. The carries are propagated through the look-ahead carry generator and applied as inputs to the second EXOR gate. After the *P* and *G* signals settle into their steady-state values, all output carries are generated after a delay of two levels of gates. Thus, outputs S_1 through S_3 have equal propagation delay times.

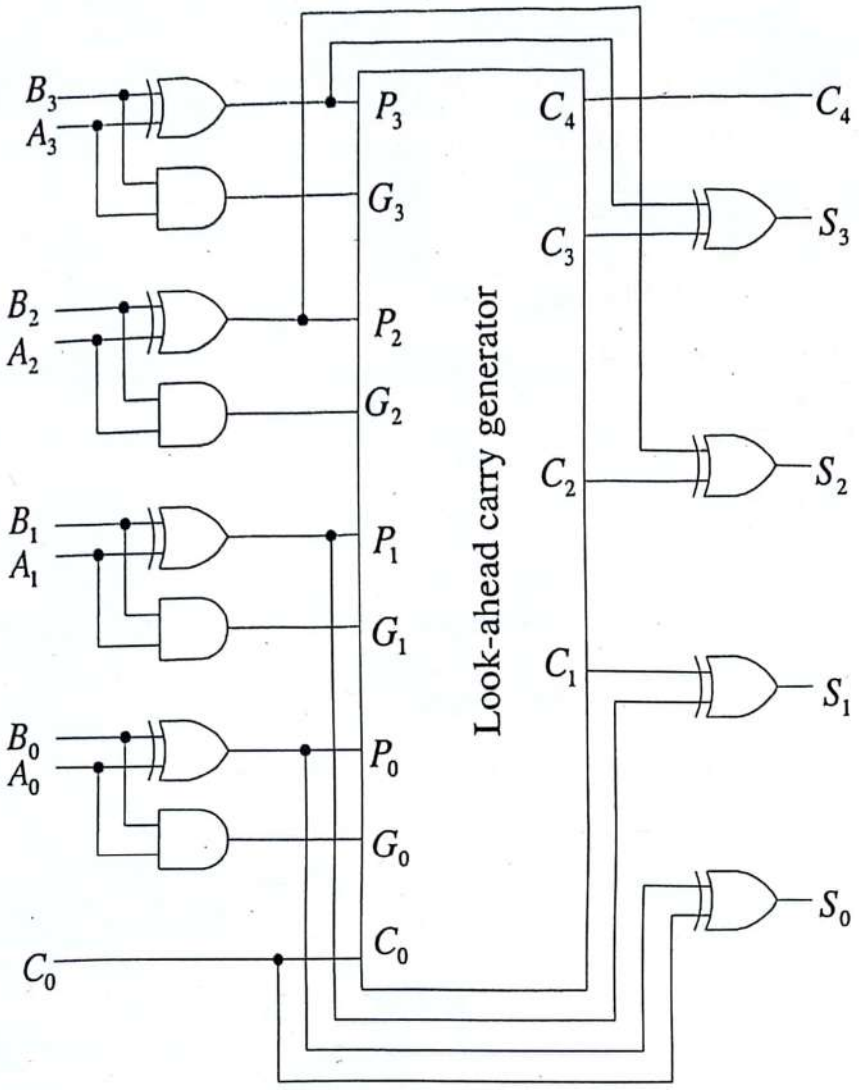


Figure 7.17 4-bit binary adder with look-ahead carry generator.

7.6 2's Complement Arithmetic Circuits

Most modern computers use 2's complement system to represent negative numbers and to perform subtraction. The operations of addition and subtraction of signed numbers can be performed using only the addition if we use the 2's complement form to represent negative numbers.

7.6.1 2's Complement Adder

Positive and negative numbers, including the sign bits, can be added together in the basic binary adder circuit of Figure 7.14 when the negative numbers are in 2's complement form. This is illustrated in Figure 7.18 for the addition of -3 and $+6$. The -3 is represented in its 2's complement form as 1101, where the first 1 is the sign bit; the $+6$ is represented in its 2's complement form as 0110, with the first 0 as the sign bit. The 4-bit binary adder produces sum outputs of 0011, which represent $+3$ in 2's complement form. The C_4 output is 1 but is disregarded in the 2's complement method.